

# Normalization of Database

---

Database Normalization is a technique of organizing the data in the database. Normalization is a systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics like Insertion, Update and Deletion Anomalies. It is a multi-step process that puts data into tabular form by removing duplicated data from the relation tables.

Normalization is used for mainly two purpose,

- Eliminating redundant(useless) data.
  - Ensuring data dependencies make sense i.e. data is logically stored.
- 

## 1.Problem Without Normalization

Without Normalization, it becomes difficult to handle and update the database, without facing data loss. Insertion, Update and Deletion Anomalies are very frequent if Database is not Normalized. To understand these anomalies let us take an example of **Student** table.

| Sid | SName  | SAddress | Subjectopted |
|-----|--------|----------|--------------|
| 401 | Adam   | Noida    | Bio          |
| 402 | Alex   | Panipat  | Maths        |
| 403 | Stuart | Jammu    | Maths        |
| 404 | Adam   | Noida    | Physics      |

---

- **Updation Anamoly** : To update address of a student who occurs twice or more than twice in a table, we will have to update **S\_Address** column in all the rows, else data will become inconsistent.
- **Insertion Anamoly** : Suppose for a new admission, we have a Student id(S\_id), name and address of a student but if student has

not opted for any subjects yet then we have to insert **NULL** there, leading to Insertion Anamoly.

- **Deletion Anamoly** : If (S\_id) 401 has only one subject and temporarily he drops it, when we delete that row, entire student record will be deleted along with it.
- 

## **2. Normalization Rule**

Normalization rule are divided into following normal form.

1. First Normal Form
2. Second Normal Form
3. Third Normal Form
4. BCNF
- 5.

### **2.1First Normal Form (1NF)**

As per First Normal Form, no two Rows of data must contain repeating group of information i.e each set of column must have a unique value, such that multiple columns cannot be used to fetch the same row. Each table should be organized into rows, and each row should have a primary key that distinguishes it as unique.

The **Primary key** is usually a single column, but sometimes more than one column can be combined to create a single primary key. For example consider a table which is not in First normal form

**Example1:**

**Student Table :**

| <b>Student</b> | <b>Age</b> | <b>Subject</b> |
|----------------|------------|----------------|
| Adam           | 15         | Biology, Maths |
| Alex           | 14         | Maths          |
| Stuart         | 17         | Maths          |

in First Normal Form, any row must not have a column in which more than one value is saved, like separated with commas. Rather than that, we must separate such data into multiple rows.

**Student Table following 1NF will be :**

| Student | Age | Subject |
|---------|-----|---------|
| Adam    | 15  | Biology |
| Adam    | 15  | Maths   |
| Alex    | 14  | Maths   |
| Stuart  | 17  | Maths   |

Using the First Normal Form, data redundancy increases, as there will be many columns with same data in multiple rows but each row as a whole will be unique.

**Example2:** Suppose a company wants to store the names and contact details of its employees. It creates a table that looks like this:

| emp_id | emp_name | emp_address | emp_mobile |
|--------|----------|-------------|------------|
| 101    | Herschel | New Delhi   | 8912312390 |
| 102    | Jon      | Kanpur      | 8812121212 |
|        |          |             | 9900012222 |
| 103    | Ron      | Chennai     | 7778881212 |
| 104    | Lester   | Bangalore   | 9990000123 |
|        |          |             | 8123450987 |

Two employees (Jon & Lester) are having two mobile numbers so the company stored them in the same field as you can see in the table above.

This table is **not in 1NF** as the rule says “each attribute of a table must have atomic (single) values”, the emp\_mobile values for employees Jon & Lester violates that rule.

To make the table complies with 1NF we should have the data like this:

| emp_id | emp_name | emp_address | emp_mobile |
|--------|----------|-------------|------------|
| 101    | Herschel | New Delhi   | 8912312390 |
| 102    | Jon      | Kanpur      | 8812121212 |
| 102    | Jon      | Kanpur      | 9900012222 |
| 103    | Ron      | Chennai     | 7778881212 |
| 104    | Lester   | Bangalore   | 9990000123 |
| 104    | Lester   | Bangalore   | 8123450987 |

---

## **2.2 Second Normal Form (2NF)**

Before we learn about the second normal form, we need to understand the following –

- **Prime attribute** – An attribute, which is a part of the prime-key, is known as a prime attribute.
- **Non-prime attribute** – An attribute, which is not a part of the prime-key, is said to be a non-prime attribute.

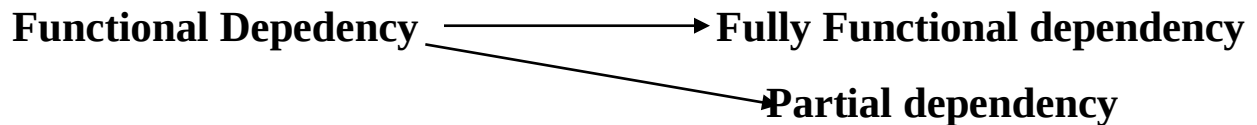
If we follow second normal form, then every non-prime attribute should be **fully functionally dependent** on prime key attribute.

## **Now What is Functional Dependency???**

A **functional dependency** occurs when one attribute in a relation uniquely determines another attribute. This can be written  $A \rightarrow B$  which would be the same as stating "B is functionally dependent upon A." **Also Known As:** Database Dependency

**Examples:** In a table listing employee characteristics including Social Security Number (SSN) and name, it can be said that name is functionally dependent upon SSN (or **SSN -> name**) because an employee's name can be uniquely determined from their SSN.

However, the reverse statement (**name -> SSN**) is not true because more than one employee can have the same name but different SSNs.



The **fully functionally dependent** mostly applies to composite primary keys. It basically means that the attribute needs to be functionally dependent on the whole primary key (not one of its parts). For example, some applications use first name, last name, and date of birth as a composite primary key. Every attribute in that record needs to depend on all first name, last name, and date of birth.

A full functional dependency occurs when you already meet the requirements for a functional dependency and the set of attributes on the left side of the functional dependency statement cannot be reduced any farther.

For example, “**{SSN, age} -> name**” is a functional dependency, but it is not a full functional dependency because you can remove age from the left side of the statement without impacting the dependency relationship.

Let R be

**NewStudent(stuId, lastName, major, credits, status, socSecNo)**

FDs in R include

- {stuId} → {lastName}, but not the reverse
- {stuId} → {lastName, major, credits, status, socSecNo, stuId}
- {socSecNo} → {stuId, lastName, major, credits, status, socSecNo}

- $\{\text{credits}\} \rightarrow \{\text{status}\}$ , but not  $\{\text{status}\} \rightarrow \{\text{credits}\}$

ZipCode  $\rightarrow$  AddressCity

- 16652 is Huntingdon's ZIP

ArtistName  $\rightarrow$  BirthYear

- Picasso was born in 1881

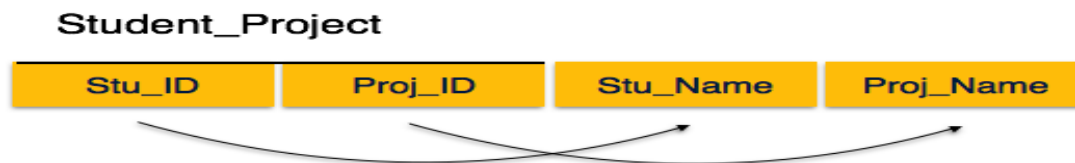
Autobrand  $\rightarrow$  Manufacturer, Engine type

- Pontiac is built by General Motors with gasoline engine

Author, Title  $\rightarrow$  PubDate

- Shakespeare's Hamlet was published in 1600

### Example1:



We see here in Student\_Project relation that the prime key attributes are Stu\_ID and Proj\_ID. According to the rule, non-key attributes, i.e. Stu\_Name and Proj\_Name must be dependent upon both and not on any of the prime key attribute individually. But we find that Stu\_Name can be identified by Stu\_ID and Proj\_Name can be identified by Proj\_ID independently. This is called **partial dependency**, which is not allowed in Second Normal Form.

### Student

|        |          |         |
|--------|----------|---------|
| Stu_ID | Stu_Name | Proj_ID |
|--------|----------|---------|

### Project

|         |           |
|---------|-----------|
| Proj_ID | Proj_Name |
|---------|-----------|

**Example2:** In example of First Normal Form there are two rows for Adam, to include multiple subjects that he has opted for. While this is searchable, and follows First normal form, it is an inefficient use of space. Also in the above Table in First Normal Form, while the candidate key is {**Student, Subject**}, **Age** of Student only depends on Student column, which is incorrect as per Second Normal Form. To achieve second normal form, it would be helpful to split out the subjects into an independent table, and match them up using the student names as foreign keys.

**New Student Table following 2NF will be :**

| <b>Student</b> | <b>Age</b> |
|----------------|------------|
| Adam           | 15         |
| Alex           | 14         |
| Stuart         | 17         |

In Student Table the candidate key will be **Student** column, because all other column i.e **Age** is dependent on it.

**New Subject Table introduced for 2NF will be :**

| <b>Student</b> | <b>Subject</b> |
|----------------|----------------|
| Adam           | Biology        |
| Adam           | Maths          |
| Alex           | Maths          |
| Stuart         | Maths          |

In Subject Table the candidate key will be {**Student, Subject**} column. Now, both the above tables qualifies for Second Normal Form and will never suffer from Update Anomalies. Although there are a few complex cases in which table in Second Normal Form suffers Update Anomalies, and to handle those scenarios Third Normal Form is there.

**Example3:** Suppose a school wants to store the data of teachers and the subjects they teach. They create a table that looks like this: Since a

teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

| <b>teacher_id</b> | <b>subject</b> | <b>teacher_age</b> |
|-------------------|----------------|--------------------|
| 111               | Maths          | 38                 |
| 111               | Physics        | 38                 |
| 222               | Biology        | 38                 |
| 333               | Physics        | 40                 |
| 333               | Chemistry      | 40                 |

Candidate Keys: {teacher\_id, subject}

Non prime attribute: teacher\_age

The table is in 1 NF because each attribute has atomic values. However, it is not in 2NF because non prime attribute teacher\_age is dependent on teacher\_id alone which is a proper subset of candidate key. This violates the rule for 2NF as the rule says “no non-prime attribute is dependent on the proper subset of any candidate key of the table”.

To make the table complies with 2NF we can break it in two tables like this:

**teacher\_details table:**

| <b>teacher_id</b> | <b>teacher_age</b> |
|-------------------|--------------------|
| 111               | 38                 |
| 222               | 38                 |
| 333               | 40                 |

**teacher\_subject table:**

| <b>teacher_id</b> | <b>subject</b> |
|-------------------|----------------|
| 111               | Maths          |
| 111               | Physics        |
| 222               | Biology        |



|     |           |
|-----|-----------|
| 333 | Physics   |
| 333 | Chemistry |

Now the tables comply with Second normal form (2NF).

---

## **2.3 Third Normal Form (3NF)**

**Third Normal form** applies that every non-prime attribute of table must be dependent on primary key, or we can say that, there should not be the case that a non-prime attribute is determined by another non-prime attribute. So this *transitive functional dependency* should be removed from the table and also the table must be in Second Normal form. For example, consider a table with following fields.

### **Transitive Functional Dependency??**

A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies. For e.g.  $X \rightarrow Z$  is a transitive dependency if the following three functional dependencies hold true:

- $X \rightarrow Y$
- $Y$  does not  $\rightarrow X$
- $Y \rightarrow Z$

Note: A transitive dependency can only occur in a relation of three or more attributes. This dependency helps us normalizing the database in 3NF (3rd Normal Form).

Example: Let's take an example to understand it better:

| <b>Book</b>        | <b>Author</b>       | <b>Author_age</b> |
|--------------------|---------------------|-------------------|
| Game of Thrones    | George R. R. Martin | 66                |
| Harry Potter       | J. K. Rowling       | 49                |
| Dying of the Light | George R. R. Martin | 66                |

{Book} ->{Author} (if we know the book, we know the author name)

{Author} does not ->{Book}

{Author} -> {Author\_age}

Therefore as per the rule of transitive dependency: {Book} -> {Author\_age} should hold, that makes sense because if we know the book name we can know the author's age.

### Student\_Detail Table :

| Student_id | Student_name | DOB | Street | city | State | Zip |
|------------|--------------|-----|--------|------|-------|-----|
|------------|--------------|-----|--------|------|-------|-----|

In this table Student\_id is Primary key, but street, city and state depends upon Zip. The dependency between zip and other fields is called **transitive dependency**. Hence to apply **3NF**, we need to move the street, city and state to new table, with **Zip** as primary key.

### New Student\_Detail Table :

| Student_id | Student_name | DOB | Zip |
|------------|--------------|-----|-----|
|------------|--------------|-----|-----|

### Address Table :

| Zip | Street | city | state |
|-----|--------|------|-------|
|     |        |      |       |

**Example2:** Suppose a company wants to store the complete address of each employee, they create a table named employee\_details that looks like this:

| emp_id | emp_name | emp_zip | emp_state | emp_city | emp_district |
|--------|----------|---------|-----------|----------|--------------|
| 1001   | John     | 282005  | UP        | Agra     | Dayal Bagh   |
| 1002   | Ajeet    | 222008  | TN        | Chennai  | M-City       |
| 1006   | Lora     | 282007  | TN        | Chennai  | Urrapakkam   |
| 1101   | Lilly    | 292008  | UK        | Pauri    | Bhagwan      |
| 1201   | Steve    | 222999  | MP        | Gwalior  | Ratan        |

Super keys: {emp\_id}, {emp\_id, emp\_name}, {emp\_id, emp\_name, emp\_zip}...so on

Candidate Keys: {emp\_id}

Non-prime attributes: all attributes except emp\_id are non-prime as they are not part of any candidate keys.

Here, emp\_state, emp\_city & emp\_district dependent on emp\_zip. And, emp\_zip is dependent on emp\_id that makes non-prime attributes (emp\_state, emp\_city & emp\_district) transitively dependent on super key (emp\_id). This violates the rule of 3NF.

To make this table complies with 3NF we have to break the table into two tables to remove the transitive dependency:

employee table:

| emp_id | emp_name | emp_zip |
|--------|----------|---------|
| 1001   | John     | 282005  |
| 1002   | Ajeet    | 222008  |
| 1006   | Lora     | 282007  |
| 1101   | Lilly    | 292008  |
| 1201   | Steve    | 222999  |

employee\_zip table:

| emp_zip | emp_state | emp_city | emp_district |
|---------|-----------|----------|--------------|
| 282005  | UP        | Agra     | Dayal Bagh   |
| 222008  | TN        | Chennai  | M-City       |
| 282007  | TN        | Chennai  | Urrapakkam   |
| 292008  | UK        | Pauri    | Bhagwan      |
| 222999  | MP        | Gwalior  | Ratan        |

---

The advantage of removing transitive dependency is,

- Amount of data duplication is reduced.
  - Data integrity achieved.
- 

## 2.4 Boyce and Codd Normal Form (BCNF)

**Boyce and Codd Normal Form** is a higher version of the Third Normal form. This form deals with certain type of anomaly that is not handled by 3NF. A 3NF table which does not have multiple overlapping candidate keys is said to be in BCNF. For a table to be in BCNF, following conditions must be satisfied:

- R must be in 3rd Normal Form
- and, for each functional dependency (  $X \rightarrow Y$  ), X should be a super Key.

Consider the following relationship : **R (A,B,C,D)**

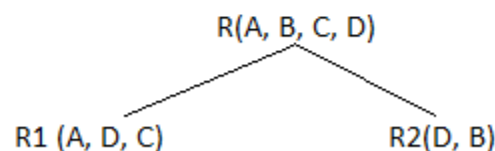
and following dependencies :

**A  $\rightarrow$  BCD**  
**BC  $\rightarrow$  AD**  
**D  $\rightarrow$  B**

Above relationship is already in 3rd NF. Keys are **A** and **BC**.

Hence, in the functional dependency, **A  $\rightarrow$  BCD**, A is the super key.  
in second relation, **BC  $\rightarrow$  AD**, BC is also a key.  
but in, **D  $\rightarrow$  B**, D is not a key.

Hence we can break our relationship R into two relationships **R1** and **R2**.



Breaking, table into two tables, one with A, D and C while the other with D and B.